



Anomaly Detection in Time Series – Clustering, Ensembles & Deep Learning

k-means Clustering for Anomaly Detection – Theory

k-means clustering offers a versatile approach to identify unusual patterns in time series data, from isolated points to extended, unexpected sequences.

k-means Clustering: The Foundation

An unsupervised algorithm, k-means partitions **n observations into k clusters**. Each observation is assigned to the cluster whose centroid (mean) it is closest to, aiming to **minimize intra-cluster distance**. The algorithm iteratively refines cluster centroids until convergence.

Cited: MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability.

Application to Anomaly Detection

Once k-means clusters the "normal" data, anomalies are identified based on their **distance from existing cluster centroids**.

- **Point Anomalies (seq=1):** Individual data points that are far from their nearest cluster centroid.
- **Discords / Subsequence Anomalies (seq>1):** Longer segments or patterns that are significantly different from any typical cluster.

Time Series Discords

A time series **discord** is defined as a subsequence that is **maximally different from all other subsequences** within the time series. These are found using a **sliding window approach**, where each subsequence within the window is compared to all others to find the most unusual one.

Cited: Keogh, E., Lin, J., & Fu, A. (2005). HOT SAX: Efficiently finding the most unusual time series subsequence. In Proceedings of the 5th IEEE International Conference on Data Mining (ICDM).

Strengths and Limitations

- **Strengths:** Simple, interpretable, and scalable for large datasets. It provides a clear way to group similar behaviors.
- **Limitations:** Highly sensitive to the initial choice of 'k' (number of clusters) and random initialization. Uses Euclidean distance, which assumes **rigid alignment** and may not capture complex time series similarities (e.g., phase shifts).

28 · hanct_kmeans() – Anomalies via k-means

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)

# k-means: Anomalias (seq=1)
dataset <- examples_anomalies$simple

model <- hanct_kmeans(1)
model <- fit(model, dataset$serie)

detection <- detect(model,
                    dataset$serie)

print(detection |>
      dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
```

```
evaluation <- evaluate(model,
                      detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##  FALSE  0 100

har_plot(model, dataset$serie,
          detection, dataset$event)
```

Step-by-step – k-means (seq=1)

- Uses `examples_anomalies$simple`
- `hanct_kmeans(1)`: k=1 cluster, detects points far from the centroid
- Observations far from the nearest centroid are flagged as anomalies
- Perfect result: 1 TP, 0 FP
- `seq=1` is for point anomalies (not sequences)

❏ **Key concept:** k-means with `seq=1` detects isolated anomalous points — observations that stray far from the normal cluster centroid. Simple and effective for point anomaly detection.

29 · hanct_kmeans() – Discords via k-means (seq>1)

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)

# k-means: Discords (seq>1)
dataset <- examples_anomalies$sequence

model <- hanct_kmeans(3)
model <- fit(model, dataset$serie)
```

```
detection <- detect(model,
  dataset$serie)

print(detection |>
  dplyr::filter(event == TRUE))
## idx event type seq seqlen
## 1 51 TRUE discord 3 3

evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
## TRUE 0 1
## FALSE 1 99

har_plot(model, dataset$serie,
  detection, dataset$event)
```

Step-by-step – k-means Discords

- Uses `examples_anomalies$sequence` (repeated patterns)
- `hanct_kmeans(3)`: k=3 clusters applied over sliding subsequences
- Windows far from their nearest centroid are flagged as discords
- Detected idx=51 (discord of length 3)
- 0 TP, 1 FP in this case — discord is close to the ground truth

📌 **Key concept:** k-means with `seq>1` detects subsequences (discords) that differ from typical recurring patterns. The choice of `seq` should reflect the expected length of the sequential anomaly.

Dynamic Time Warping (DTW) – Theory

Dynamic Time Warping (DTW) provides a powerful alternative to traditional distance metrics for time series analysis, specifically designed to handle temporal discrepancies.

What is DTW & How it Works

DTW is a distance measure that allows flexible temporal alignment between two sequences, unlike Euclidean distance which requires strict point-to-point correspondence. It finds the optimal "warping path" between two series by minimizing the total distance, effectively stretching or compressing one sequence in time to match the other.

Cited: Sakoe, H., & Chiba, S. (1978). Dynamic programming algorithm optimization for spoken word recognition. IEEE Transactions on Acoustics, Speech, and Signal Processing.

DTW for Anomaly Detection

DTW excels in scenarios where time series exhibit **phase shifts, speed variations, or local distortions**, which Euclidean distance struggles to accommodate. In anomaly detection, DTW-based clustering assigns subsequences to clusters; those with a large DTW distance to their nearest cluster centroid are flagged as anomalies or discords.

Cited: Berndt, D. J., & Clifford, J. (1994). Using dynamic time warping to find patterns in time series. In KDD Workshop.

Implementation & Considerations

For practical applications in R, the `dtwclust` package is commonly used to implement DTW-based clustering and anomaly detection. While DTW is highly robust to temporal distortions, its primary limitation is its **higher computational cost** compared to simpler distance measures like Euclidean distance.

 DTW is particularly useful when the **shape** of the time series is more important than its precise timing.

30 · hanct_dtw() – Anomalies via DTW

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)

# DTW: Anomalias (seq=1)
dataset <- examples_anomalies$simple

model <- hanct_dtw(1)
model <- fit(model, dataset$serie)
## Found more than one class "dist"
## in cache; using the first...
```

```
detection <- detect(model,
  dataset$serie)

print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly

evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
## TRUE 1 0
## FALSE 0 100

har_plot(model, dataset$serie,
  detection, dataset$event)
```

Step-by-step – DTW Anomalies

- DTW (Dynamic Time Warping) measures distance with flexible temporal alignment
- The warning about class "dist" is expected (namespace conflict)
- Implementation via `dtwclust`; thresholds computed via `harutils()`
- Perfect result: 1 TP, 0 FP
- More robust than Euclidean distance for distorted or phase-shifted patterns

📌 **Key concept:** DTW enables comparison of subsequences with temporal shifts — more robust than Euclidean distance for patterns that vary in speed or phase.

31 · hanct_dtw() – Discords via DTW (seq=3)

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)

# DTW: Discords (seq=3)
dataset <- examples_anomalies$sequence

model <- hanct_dtw(3)
model <- fit(model, dataset$serie)
```

```
detection <- detect(model,
  dataset$serie)

print(detection |>
  dplyr::filter(event == TRUE))
## idx event type seq seqlen
## 1 52 TRUE discord 3 3

evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
## TRUE 0 1
## FALSE 1 99

har_plot(model, dataset$serie,
  detection, dataset$event)
```

Step-by-step – DTW Discords

- Sliding windows are compared to the nearest centroid using DTW distance
- Windows with a large DTW distance are flagged as discords
- Detected idx=52 (close to the ground truth at idx=50)
- 0 TP, 1 FP — offset of 2 steps from the ground truth
- DTW is more robust than k-means for distorted or phase-shifted patterns

 **Key concept:** DTW for discord detection compares subsequences with flexible temporal alignment. Ideal for series with repeating patterns that vary in speed or phase — more robust than Euclidean k-means.

Ensemble Methods for Anomaly Detection – Theory

Ensemble methods are a powerful approach in anomaly detection, leveraging the collective intelligence of multiple base detectors to achieve more robust and accurate results than any single detector could alone.

Combining Detectors

Ensemble methods operate by combining predictions from several individual anomaly detectors. This integration process typically yields a final decision that is less prone to errors and more reliable, making the detection system more resilient to noise and subtle anomalies.

Motivation & Robustness

Individual anomaly detectors often possess distinct strengths and weaknesses. By combining them, an ensemble can mitigate the limitations of any single model, significantly reducing false positives and enhancing the overall robustness against diverse types of anomalies present in time series data.

Majority Voting

A common strategy for combining detector outputs is majority voting. In this scheme, a specific data point is flagged as an anomaly only if a predefined majority of the constituent base detectors independently identify it as anomalous, thus requiring strong consensus for a positive detection.

Detector Diversity

The effectiveness of an ensemble heavily relies on the diversity of its base detectors. Combining models from different families—such as regression-based (e.g., ARIMA), spectral (e.g., EMD), and statistical (e.g., FBIAD)—is crucial, as each brings a unique perspective to anomaly identification. This ensures a comprehensive coverage of anomaly types.

Bias-Variance Tradeoff

A key advantage of diverse ensembles is their ability to reduce the variance of the overall prediction, which translates to fewer false positives, without unduly increasing bias. This balance contributes to more stable and accurate anomaly detection, especially in complex time series datasets.

32 · har_ensemble() – Majority Voting

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

har_plot(harbinger(), dataset$serie)

# Definir ensemble de detectores base
# FBIAD + ARIMA + EMD votam juntos
model <- har_ensemble(
  hanr_fbiad(),
  hanr_arima(),
  hanr_emd())

# Ajustar todos os detectores base
model <- fit(model, dataset$serie)

# Detectar via votação por maioria
detection <- detect(model,
  dataset$serie)

print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

har_plot(model, dataset$serie,
  detection, dataset$event)

har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step by Step

- `har_ensemble()` accepts any number of base detectors
- Each base detector casts one vote
- The final event reflects majority consensus
- Base detectors: FBIAD, ARIMA, and EMD — three distinct method families
- Perfect result: 1 TP, 0 FP
- Combining methods from different families reduces false positives

 **Key concept:** Majority voting ensembles combine multiple detectors from different families. A point is flagged only if the majority of detectors agree — this reduces the false positives produced by any single detector.

Temporal Fuzzification in Ensembles – Theory

In ensemble anomaly detection, individual base detectors often identify the same underlying anomaly but with slight temporal shifts. Temporal fuzzification is a crucial technique designed to address this challenge, ensuring robust consensus and more precise anomaly reporting.

The Temporal Offset Challenge

Different detectors in an ensemble may flag the same real anomaly at slightly different time steps. This can lead to duplicate detections being counted or, conversely, prevent the ensemble from reaching a strong consensus if individual votes are too dispersed in time. It complicates accurate evaluation and interpretation.

Defining Temporal Fuzzification

Temporal fuzzification is a technique that merges individual anomaly detections that occur within a defined time window (known as a `time_tolerance`) before the ensemble's voting rule is applied. It transforms exact timestamps into temporal neighborhoods.

Mechanism of Merging

Before the final voting stage, if two or more base detectors flag events within the specified `time_tolerance` window, these separate flags are consolidated. They are treated as a single, unified event from that specific detector for the purpose of ensemble voting, simplifying the input to the consensus mechanism.

Benefits: Precision & Alignment

This approach prevents the double counting of a single anomaly by multiple detectors, which significantly improves the precision of the ensemble's output. It also ensures that the detected anomalies align more accurately with the true granularity of ground truth labels, where an anomaly is typically considered a singular event, not a series of staggered flags.

When to Employ Fuzzification

Temporal fuzzification is particularly beneficial when base detectors are known to exhibit slight temporal offsets in their anomaly identifications. For instance, if a detector like ARIMA consistently identifies an event two steps after its actual occurrence, this method helps to harmonize its output with other detectors or ground truth.

Cited: Tatbul, N., Lee, T. J., Zdonik, S., Alam, M., & Gottschlich, J. (2018). Precision and recall for time series. In Advances in Neural Information Processing Systems (NeurIPS). | Gama, J., Zliobaite, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. ACM Computing Surveys.

33 • har_ensemble() Fuzzy – Temporal Tolerance

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

har_plot(harbinger(), dataset$serie)

# Definir ensemble com fuzzificação
model <- har_ensemble(
  hanr_fbiad(),
  hanr_arima(),
  hanr_emd())

# Tolerância temporal: detecções
# dentro de 10 passos são fundidas
model$time_tolerance <- 10

model <- fit(model, dataset$serie)

detection <- detect(model,
  dataset$serie)

print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##  FALSE  0 100

har_plot(model, dataset$serie,
  detection, dataset$event)

har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step by step

- Same setup as the basic ensemble
- Add model\$time_tolerance <- 10 after creating the ensemble
- Detections within 10 time steps are merged before voting
- Reduces duplicate triggers from detectors that disagree slightly in time
- Better aligns with the granularity of the ground truth
- Result: 1 TP, 0 FP (same as the simple ensemble here)

❏ **Key concept:** Temporal fuzzification merges nearby detections before voting. Useful when different detectors flag the same event at slightly different moments — prevents double counting.

Summary: Method Families and When to Use Them

Clustering: k-means

`hanct_kmeans()`

Use when you want to detect point anomalies (`seq=1`) or discords (`seq>1`) based on distance to cluster centroids. Simple and effective for pattern-based detection.

Clustering: DTW

`hanct_dtw()`

Use when subsequences may vary in speed or phase. DTW provides flexible temporal alignment — more robust than Euclidean distance for distorted or shifted patterns.

Ensembles: Majority Voting

`har_ensemble()`

Use when you want to combine multiple detectors from different families (e.g., FBIAD, ARIMA, EMD). A point is flagged only if the majority agrees — reduces false positives.

Ensembles: Temporal Fuzzification

`har_ensemble()` + `time_tolerance`

Use when detectors from the same ensemble flag the same event at slightly different time steps. Merges nearby detections before voting — prevents double counting.

Additional Sources

Explore the resources below to go deeper into the methods covered in this deck: k-means and DTW clustering, ensemble voting, and the Harbinger framework.



CRAN Package

cran.r-project.org/web/packages/harbinger



Event Detection Resources

eic.cefet-rj.br/~eogasawara/en/event-detection-in-time-series



DAL Toolbox

cran.r-project.org/web/packages/daltoolbox



dtwclust Package

cran.r-project.org/web/packages/dtwclust

R package used internally by `hanct_dtw()` for Dynamic Time Warping clustering.



Issues & Discussions

github.com/cefet-rj-dal/harbinger/issues

Open questions, report bugs, or suggest improvements directly in the repository.